

AutoTune 设计文档

 已关联4个iCafe卡片

i Autotune 的终极目标是在运行时对不同的输入 pattern 选择不同的运行策略，达到最优性能。

ZHQ:

AT 的使用场景应当是：

- 线下长时间运行，得出各个 pattern 对应的最优策略。
- 线上直接检索，利用检索结果调用 kernel。

所以其设计目标应该是：

- 检索一定要快（树结构，数据限制在 4K 页内，尽可能连续或根据 cacheline 的长度设计树层次；再或者就一个 array 顺序查）
 - 也不一定要快，想到 cpu 空间还不少(昆仑1直接用一个map暴力做)
- 线下 profiling 的时候时间可以久一些，但保存的 profiling 数据应该泛用性强，足以应对不同的输入数据 pattern，对未测试的 pattern 有一定的预测能力(常见需求是batchsize、图像的h和w可能变化，但没有被tune到)

如果可以真的应该 train 个模型.....

先 train 个模型，然后利用模型生成策略文档.....

策略文档如何设计，理想情况应该是 $x [a, b] y [c, d] z[e, f] \rightarrow \text{kernel}[0]$ 这样？

nehw 会变，其他参数不变。能 hold 住 nehw 的改变也可以。

难点：

1. 如何让目前点对点的 autotune 输出文件能在一定限制下(例如只有 nhw 改变)的参数变动中依然发挥作用（性能 $\geq$ planner）。
 - a. 输出文件的格式

TODO:

1. ~~重构 selector~~ 进行模版化
2. 实现序列化反序列化

3. ~~main-loop~~ 模版函数
4. ~~choices-generator~~ 模版函数
5. ~~autotune-all-in-one~~ 模版函数
6. 实现成 class 自己维护 data holder ?

SJX:

步骤:

数据结构叫『algorithm selector』，本质上是要同时支持autotune和非autotune的场景。

autotune的阶段分成如下几步:

1. proposal: 提出数目合适的分块方案、执行策略等等
7. filter: 移除不可运行的方案
8. run: 运行所有合适的方案，选择出最佳的，存到某个数据结构里

非autotune的步骤?

需要tune的点?

分块大小、L3怎么用?

有的卷积涉及到难搞的参数，如dilation，可能会有多种实现方案，比如：im2im+conv、im2col+fc

输入格式:

参数、指针是L3还是GM、可用的L3的量

数据结构需要提供的接口可以先考虑一下

XXT:

planner这边提供一个接口，用来判断n, h, w变化时，是否可以复用plan。

是否复用的判断标准有两个:

1. 是否满足sdnn硬件的各种限制
2. 是否性能不会下降的太厉害（例如DMA不对齐）

类似:

```
template <int formart, typename TX, typename TW, typename TY, typename TGEMM>
bool reuse_plan(int n, int h, int w, int new_n, int new_h, int new_w, XPU2ConvPlan_t& plan);
```

复用plan有一个挑战是：应该无法保证复用的plan一定比planner建模算出来的plan更优

5.31 考虑快速实现

提供如下接口：

```
</> C++ | 收起 ^

1 -----
2 // 6_13 更新
3 template<typename PARAM_T, typename PLAN_T>
4 int k12_autotune(Context* ctx, PARAM_T params, int autotune_loop,
5     std::vector<PLAN_T> (*generate_choices)(PARAM_T),
6     int (*autotune_kernel_wrapper)(Context*, PARAM_T, PLAN_T), &PLAN_T) {
7
8 // 目前将 1 2 接口合并为一个
9 std::vector<PLAN_T> (*generate_choices)(PARAM_T) // 生成经过筛选的 choices, 开发者要保证 choices 的合法性, 并且控制产生的 choices 的数量。
10
11 // 调用kernel, 内部可能需要分配内存, kernel运行成功会返回api::SUCCESS
12 int (*autotune_kernel_wrapper)(Context*, PARAM_T, PLAN_T) //和原来一样
13 -----
14 // range_start, range_end, autotune_step
15 // 循环的范围是[range_start, range_end), 即 >= range_start, < range_end
16 typedef tuple<int, int, int> conv_autotune_range_t; // 应该和autotune内部的定义一致
17 std::vector<conv_autotune_range_t> get_autotune_range(conv_fwd_param_t param); // 给出所有需要排列组合的情况
18
19 // 筛选有效的plan: 返回true, 并且补充计算plan的不参与tune的选项
20 bool skip_or_complete_plan(conv_fwd_param_t param, XPU2ConvPlan_t& plan);
21
22 // 调用kernel, 内部可能需要分配内存, kernel运行成功会返回api::SUCCESS
23 template<typename TX, typename TW, typename TY, typename TGEMM>
24 int launch_conv_fwd_sdnn(Context* ctx, conv_fwd_param_t param, XPU2ConvPlan_t plan);
25
26
27 // ----- TODO -----
28 template<typename TX, typename TW, typename TY, typename TGEMM>
29 xpu2_conv_plan_key_t generate_key(conv_fwd_param_t param); // 这个接口可能会不需要? 看haoqi的autotune内部存储
30
31 // 存储autotune生成的plan, haoqi可能不用, 由wrapper来调用更合理
32 void insert_plan(conv_fwd_param_t param, const XPU2ConvPlan_t& plan);
33
```

TODO: 是否需要考虑cluster_num? cluster_num应该template参数?

ZLF:

我这边可以先整理反向这边对输入输出的要求，考虑『预测的pattern出来的结果是否满足限制、性能是否下降厉害』的接口。

对【L3 size拆解成一个个处理过的变量】作为key值，有顾虑 L3策略的迭代是否会频繁更换autotune对外的接口。

AUTOTUNE 序列化反序列化实现

目前 kl2 autotune 对 结果的保存与 reload 依然通过简单的 map 序列化与反序列化实现。

较为不同的设计在于：

为了将 kernel 的 plan 和对应的 序列化反序列化放在一起，降低后续维护的开销，直接将 to_string 和 set_from_string 定义在结构体中。

```
You, an hour ago | 2 authors (You and others)
21  struct XPU2ConvPlan_t {
22      int f_split;
23      int h_split;
24      int c_tile;
25      int f_tile;
26      int yh_tile;
27      int yw_tile;
28      int l2_c_subtile;
29      int l2_yh_subtile;
30      int f_subtile;
31      int yhw_subtile;
32      int reuse_filter;
33      int need_GM_temp_y;
34      bool y_use_l3_opt;    // use l3 optimization in wrapper
35      int l3_opt_n_tile;
36
37  > std::string to_string() {
55
56  > void set_from_string(std::string line) {
73  };
74
```

kl2 conv selector 头文件

如此，在 reload 和 save 的时候，就可以很方便的实现数据的存取，且不易产生 bug。

```

};
// autotune file layout:
// === file head ===
// PARAM_T
// PLAN_T
// PARAM_T
// PLAN_T
// ...
// === file tail ===
while (getline(infile, line)) {
    xpu2_conv_plan_param_t params = plan_string_to_tuple(line);
    getline(infile, line);
    XPU2ConvPlan_t plan;
    plan.set_from_string(line);
    conv_planner_map[params] = plan;
    valid_cnt++;
}
infile.close();
std::cout << "KL2 autotune_file{" << filename << "} set " << valid_cn

```

kl2 autotune 数据 reload 代码

比较理想的情况是对 map 的 key(PARAM_T) 和 value(PLAN_T) 都使用这种方式。

但目前 PARAM_T 需要作为 std::map 的 key，则其需要实现小于号的重载以满足 map 的排序要求。

为了方便开发，目前 param 没有抽象成数据结构，而是使用的 std::tuple。

所以无法像 PLAN 一样将序列化反序列化与结构体本身放在一起。

目前的实现是在对应 selector 的 autotune 文件存取函数中，通过 lambda 函数实现两个将 tuple 和 str 互相转化，该实现可以 work，代码还算简洁，但是后续如果更新位于 selector.h 头文件中的 param tuple 的时候，存在忘记更新反序列化函数的可能性，是个坑，

```

059 |     std::ofstream outfile;
060 |     outfile.open(autotune_filename);
061 |
062 | > auto plan_tuple_to_string = [&](xpu2_conv_plan_param_t plan)-> std::string{
074 |
075 |     int valid_cnt = 0;
076 |     for (auto it = conv_planner_map.begin(); it != conv_planner_map.end(); ++it) {
077 |         auto key = it->first;
078 |         auto value = it->second;
079 |         outfile << plan_tuple_to_string(key) << std::endl;
080 |         // tuple to string
081 |         outfile << value.to_string() << std::endl;
082 |         valid_cnt++;
083 |     }
084 |     std::cout << "KL2 autotune_file{" << autotune_filename << "} save " << valid_cnt << " records"<< std
085 |     outfile.close();
086 | }
087 |

```

You, 31 minutes ago | 1 author (You)

```

088 | > template<typename TX, typename TW, typename TY, typename TGEMM> struct post_type_trait {

```

kl2 autotune 数据保存

AUTOTUNE MAIN LOOP 的实现

```

4
5 namespace baidu { namespace xpu { namespace api {
6
7 template<typename PLAN_T, typename PARAM_T, typename DATA_PACK_T>
8 int autotune_looper(Context* ctx, int (*autotune_kernel_wrapper)(Context*, PLAN_T, PARAM_T, DATA_PACK_T),
9     std::vector<PLAN_T> choices, PARAM_T params, int autotune_loop, DATA_PACK_T data_pack) {
10
11     int ret = 0;
12     uint32_t best_avg = 1 << 30;
13     int best_idx = 0;
14     xpu_wait(ctx->xpu_stream);
15     for (int choice_idx = 0; choice_idx < choices.size(); ++choice_idx) {
16         struct timeval start;
17         struct timeval end;
18
19         gettimeofday(&start, NULL);
20         for (int i = 0; i < autotune_loop; ++i) {
21             // xpu_wait(ctx->xpu_stream);
22             ret = autotune_kernel_wrapper(ctx, choices[i], params, data_pack);
23             // xpu_wait(ctx->xpu_stream);
24         }
25         xpu_wait(ctx->xpu_stream);
26         gettimeofday(&end, NULL);
27         uint32_t diff = 1000000 * (end.tv_sec - start.tv_sec) + end.tv_usec - start.tv_usec;
28         if (diff / autotune_loop < best_avg) {
29             best_avg = diff / autotune_loop;
30             best_idx = choice_idx;
31         }
32     }
33     return best_idx;
34 }
35
36 }}}

```

autotune main loop

由于 autotune 的主要 tune 的过程是比较固定的，因此可以直接抽象成一个 autotune_looper 的模版函数，定义在 **autotune_common.h** 头文件中。

模版参数：

1. PLAN_T 各个算子作为 plan 的数据结构。（对 conv2d 来说就是 `XPU2ConvPlan_t`
2. PARAM_T 新定义的 param_t，要求 op 定义自己的 param_t，该类型应当包含除 PLAN_T 外的所有运行一个 kernel 所需要的变量。

AUTOTUNE 模版函数的调用接口

// 详情参考: http://agroup.baidu.com/kunlun_software_doc/md/article/4894006

```

template<typename PARAM_T, typename PLAN_T>
PLAN_T kl2_autotune(Context* ctx, PARAM_T params, int autotune_loop,
    std::vector<PLAN_T> (*generate_choices)(PARAM_T),
    int (*autotune_kernel_wrapper)(Context*, PARAM_T, PLAN_T)) {
    std::vector<PLAN_T> choices = generate_choices(params);
    std::cout << "autotune_gen_choices get choices: {" << choices.size() << "}" << std::endl;

    int best_idx = 0;
    best_idx = autotune_looper<PARAM_T, PLAN_T>(ctx, autotune_kernel_wrapper, choices, params, autotune_loop);
    std::cout << "autotune pick best_idx: " << best_idx << " plan: " << choices[best_idx].to_string() << std::endl;
    // TODO data holder
    return choices[best_idx];
}
}}}

```

You, 3 days ago • kunlun-api-1206 autotune framework init

#--116

autotune 调用接口

http://agroup.baidu.com/kunlun_software_doc/md/article/4894006 具体使用方法可参考该文档。

需要 owner 帮忙实现的函数

1. 定义 plan 数据结构，包含描述一个 plan 的所有参数。同时，在 plan 内部实现 **to_string**, **set_from_string** 这两个函数。图中的 **init_from_vec** 是历史残留，后续可以删除。

```
100, 3 days ago | 2 authors (xiexiating and others)
struct XPU2ConvPlan_t {
    int f_split;
    int h_split;
    int c_tile;
    int f_tile;
    int yh_tile;
    int yw_tile;
    int l2_c_subtile;
    int l2_yh_subtile;
    int f_subtile; xiexiating, 7 months ago * kunlun-api-385
    int yhw_subtile;
    int reuse_filter;
    int need_GM_temp_y;
    bool y_use_l3_opt; // use l3 optimization in wrapper
    int l3_opt_n_tile;

    bool init_from_vec(const std::vector<int>& algo) { ...
}
std::string to_string() { ...
}
void set_from_string(std::string line) { ...
};
```

2. 定义 param 结构，现为一个 `std::tuple` 的 typedef。param 结构应包含所有运行一个 kernel 所需要的包含数据的所有参数。

```
131 struct xpu2_conv_fwd_param_t {
132     Dtype TX;
133     Dtype TW;
134     Dtype TY;
135     Dtype TGENM;
136     int n;
137     int c;
138     int xh;
139     int xw;
140     int fh;
141     int kh;
142     int kw;
143     int sh;
144     int sw;
145     int pad_up;
146     int pad_down;
147     int pad_left;
148     int pad_right;
149     const void* x;
150     const void* weight; xiexiating, 3 days ago * kunlun-api-1189 [Tech Tas
151     void* y;
152     const float* x_maxptr;
153     const float* w_maxptr;
154     const float* branch_maxptr;
155     float* y_maxptr;
156     const float* bias;
157     const void* branch;
158     float* act_ew_ptr;
159     uint32_t act_ew_packed_id;
160     bool is_nchw;
161     int l3_size;
162     int cluster_num;
163
164     std::vector<std::pair<const void*, int>> checkpoint_datas() {
165         int yh = (xh + pad_up + pad_down - kh) / sh + 1;
166         int yw = (xw + pad_left + pad_right - kw) / sw + 1;
167         return { std::make_pair(branch, n * f * yh * yw * Dtype_size(TY)) };
168     }
169 };
```

3. 定一个 key 结构，为 2 中 param 结构的子集。key 结构包含所有与 plan 相关的 param，作为存储数据结构的 key 存在。

```
xiexiating, 9 days ago | 1 author (xiexiating)
typedef std::tuple<Dtype, Dtype, Dtype, Dtype, Dtype, int, int, int, int, int, int,
// stride_h, stride_w, pad[4], is_nchw, has_branch_or_not, use_l3_bitmap, l3_size
int, int, int, int, int, int, bool, bool, int, int> xpu2_conv_fwd_key_t;
```

4. 实现 **gen_choices** 函数如下格式，要求输入是 2 中定义的 param 类型，输出是一个 1 中定义的 plan 的 vector，要求控制 plan 的数量（删除明显时间过长的plan），且生成的 plan 搭配输入的 param 一定能正常调用 kernel。

</>

C++ | 收起 ^

```
1 std::vector<PLAN_T> (*generate_choices)(PARAM_T),
```

5. 实现一个 kernel_wrapper 如下格式，要求输入是 ctx， param 和 plan， 能保证运行一个 kernel。

</>

C++ | 收起 ^

```
1 int (*autotune_kernel_wrapper)(Context*, PARAM_T, PLAN_T)
```

实现 DEMO

```
template<typename TX, typename TW, typename TY, typename TGEMM>
static XPU2ConvPlan_t dlgl_sdn_get_plan(Context* ctx, int n, int c, int h, int w, int f,
    const std::vector<int>& ksize, const std::vector<int>& stride, const std::vector<int>& pad,
    const std::vector<int>& dilation, int group, const TX* x, const TW* weight, TY* y,
    const float* x_maxptr, const float* w_maxptr, const float* branch_maxptr, float* y_maxptr,
    const float* bias, const TY* branch, const Activation_t& act, bool is_nchw,
    float* ew_ptr, uint32_t ew_packed_id) {
    int l3_size = ctx->l3_mgr.get_size();
    const int cluster_num = ctx->nsdn();
    auto param = std::make_tuple(CPPTTypeToDtype<TX>(), CPPTTypeToDtype<TW>(), CPPTTypeToDtype<TY>(),
        CPPTTypeToDtype<TGEMM>(), n, c, h, w, f, ksize[0], ksize[1], stride[0], stride[1],
        pad[0], pad[1], pad[2], pad[3], x, weight, y, x_maxptr, w_maxptr, branch_maxptr,
        y_maxptr, bias, branch, ew_ptr, ew_packed_id, is_nchw, l3_size, cluster_num);
    int autotune_loop = ctx->xpu2_conv_selector.get_autotune_loop();
    XPU2ConvPlan_t plan;
    if (autotune_loop <= 0) {
        plan = ctx->xpu2_conv_selector.conv_fwd_make_plan(param);
    } else {
        if (ctx->xpu2_conv_selector.conv_fwd_find_plan(param, plan)) {
            return plan;
        }
        auto generate_func = &XPU2ConvAlgoSelector::conv_fwd_generate_choices;
        plan = kl2_autotune<xpu2_conv_fwd_param_t, XPU2ConvPlan_t>(ctx, param, autotune_loop,
            generate_func, &launch_conv_fwd_sdn<TX, TW, TY, TGEMM>);
        ctx->xpu2_conv_selector.conv_fwd_insert_plan(param, plan);
    }
    return plan;
}
```

xiexiating, 3 days ago • kunlun-api-